CHUYÊN ĐỀ BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

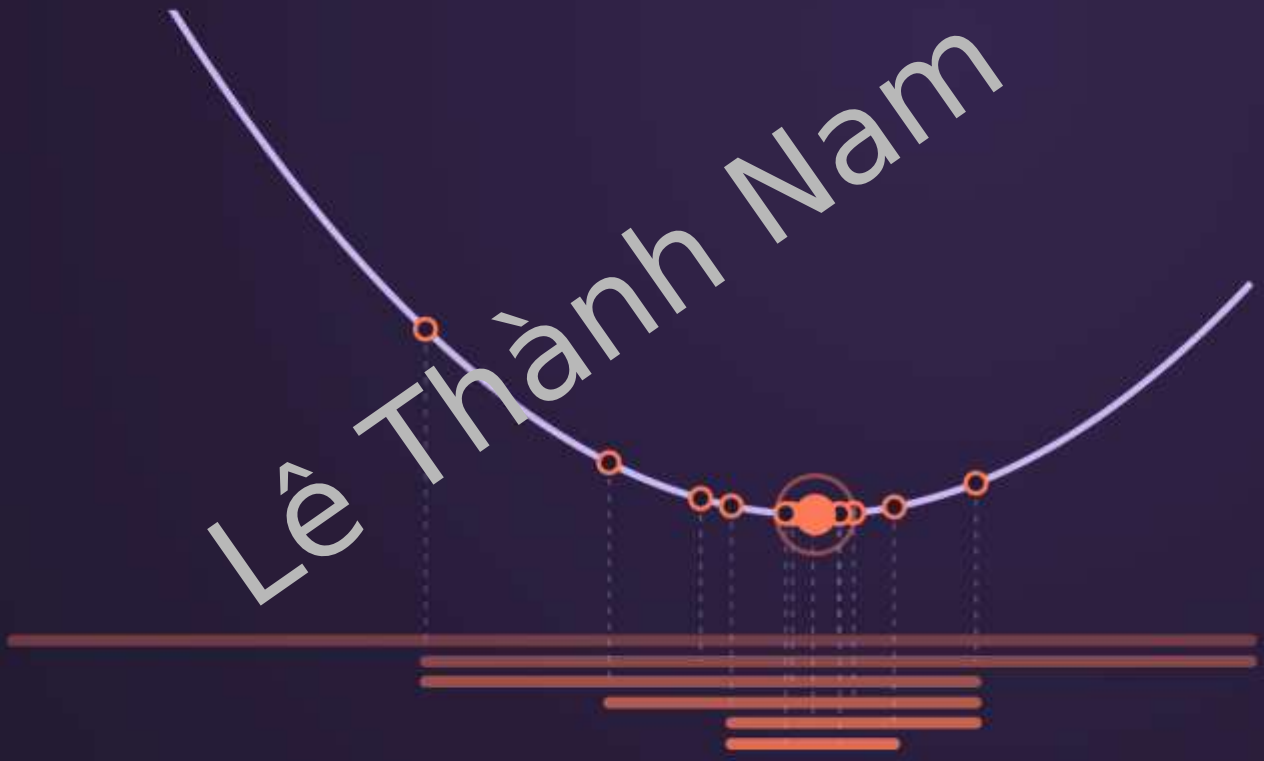
三分

TAM PHÂN

CƠ BẢN & NÂNG CAO

qua 12 bài Luogu 洛谷

Lê Thành Nam



Mỗi vạch cam là đoạn còn nghi ngờ; hai chấm là m1, m2. So sánh f(m1) với f(m2), bỏ một phần ba, cho tới khi chạm đáy của hàm lồi.

```
if (f(m1) > f(m2)) l = m1;
else r = m2;
```

4 chương · 12 bài · 30 chương trình đã kiểm chứng và đo thời gian thật

1 Tam phân số thực | 2 Tam phân số nguyên | 3 Lồng nhau · đối ngẫu | 4 WQS · tính lồi

LÊ THÀNH NAM

tính lồi · ⅓ · ⅔

MỤC LỤC

MỤC LỤC 1
GIỚI THIỆU 2
Chương 1. Tam phân trên hàm số thực 4
Bài 1. Tìm đỉnh của hàm một đỉnh (bài mẫu) 5
Bài 2. Cực tiểu của hàm max (bài mẫu) 8
Bài 3. Độ yếu và độ nghèo 11
Chương 2. Tam phân trên số nguyên 15
Bài 4. Hai anh em 16
Bài 5. San bằng các cột gạch 19
Bài 6. Kì thi cuối kì 23
Chương 3. Tam phân lồng nhau, đối ngẫu và con trỏ 27
Bài 7. Băng chuyền 28
Bài 8. Điểm hẹn 31
Bài 9. Lớn nhất trừ trung bình 34
Chương 4. Tính lồi theo số lượng: chặt nhị phân hệ số phạt (WQS) 38
Bài 10. Cây khung với đúng k cạnh trắng 39
Bài 11. Đi săn Pokémon 43
Bài 12. Đặt bưu điện (bản tăng cường) 47
PHỤ LỤC 52

Lê Thành Nam

GIỚI THIỆU

Cuốn sách này dành cho ai

Sách gồm 12 bài tam phân và tối ưu hàm lồi chọn từ kho đề Luogu (洛谷, luogu.com.cn), gồm cả các bài Codeforces được Luogu lưu trữ (mã bắt đầu bằng CF). Các bài được xếp thành 4 chương: tam phân trên số thực, tam phân trên số nguyên, tam phân lồng nhau cùng các cách nhìn đối ngẫu, và kĩ thuật chặt nhị phân hệ số phạt (WQS), một "người họ hàng" của tam phân dựa trên tính lồi theo số lượng.

Sách phù hợp với học sinh đã quen chặt nhị phân. Bài dễ nhất là các bài mẫu (模板); bài khó nhất kết hợp tam phân hoặc WQS với quy hoạch động, cây khung và tối ưu quyết định đơn điệu.

Mỗi bài gồm hai phần. **Đề bài** được viết lại bằng tiếng Việt, giữ đúng yêu cầu, ràng buộc, định dạng vào/ra và ví dụ của đề gốc. **Hướng dẫn giải** đi qua nhiều tầng, từ ngây thơ đến tối ưu, kèm code chạy được, thời gian đo thật và hình minh hoạ. Đặc biệt, mỗi bài đều có lập luận vì sao hàm cần tối ưu là lồi (hoặc một đỉnh): đó là điều kiện để tam phân đúng.

> **Về cách viết đề**
>
> Đề trong sách không phải bản dịch từng câu. Mỗi đề được viết lại theo cách hiểu tự nhiên của người Việt, bỏ những chi tiết kể chuyện không ảnh hưởng đến lời giải. Mọi điều kiện, giới hạn, định dạng vào/ra và test ví dụ được giữ nguyên; mọi ví dụ đã được đối chiếu bằng cách chạy lời giải. Giới hạn thời gian ghi trong sách là giới hạn sách dùng để chấm các subtask, có thể khác giới hạn trên Luogu.

Cách giải

Hướng dẫn giải của mỗi bài chia thành các phần có dải màu riêng; hầu hết các bước đều có hình minh hoạ.

Phần	Nội dung
HIỂU ĐỀ	Vẽ ra các trường hợp để hiểu chính xác yêu cầu; nêu các nhận xét đầu tiên, thường là hàm cần tối ưu và tính lồi của nó
NGÂY THƠ	Tầng 1: ý tưởng tự nhiên nhất (thử mọi giá trị, chia lưới, quy hoạch động đủ chiều), code và độ phức tạp
BẾ TẮC	Thời gian chạy đo thật của tầng 1, chỉ ra nguyên nhân chậm, kết thúc bằng một *câu hỏi then chốt*
CẢI TIẾN	Các tầng trung gian, có cả những tầng "nhanh nhưng sai": chia lưới thiếu chính xác, xử lí sai đoạn nằm ngang
LÓE SÁNG	Nhận xét quyết định: chứng minh tính lồi, cách nhìn đối ngẫu, kĩ thuật phạt (WQS)
TỐI ƯU	Code tối ưu, độ phức tạp và bảng so sánh tất cả các tầng trên mọi subtask
LƯU Ý	Những chỗ dễ sai và phần mở rộng

Mỗi bài có ít nhất một hộp **Thiết kế lời giải** màu vàng, tóm tắt hàm cần tối ưu, lí do nó lồi và khuôn tam phân (hoặc WQS) được dùng.

Thời gian chạy trong sách được đo thật bằng trình biên dịch g++ 13 (tuỳ chọn -O2, chuẩn C++17) trên máy chủ Linux, tính cả thời gian đọc và ghi dữ liệu. Trên máy chấm Luogu các con số có thể khác đôi chút, nhưng tỉ lệ giữa các tầng là như nhau.

Subtask và các tầng lời giải

Mỗi bài có một bảng subtask: subtask đầu dành cho lời giải ngây thơ, subtask cuối là ràng buộc gốc của Luogu (hoặc mở rộng, có ghi rõ). Cuối mỗi bài có bảng so sánh: mỗi tầng chạy trên test lớn nhất của từng subtask, ghi thời gian thật và số điểm đạt được.

Lê Thanh Nam

Sử dụng sách

Với mỗi bài, nên đọc đề, tự nghĩ ít nhất 20–30 phút rồi mới đọc hướng dẫn, và dừng lại ở mỗi "Câu hỏi then chốt" để tự trả lời trước khi đọc tiếp. Mọi code trong sách đã được chạy với ví dụ của đề và đối chiếu chéo với nhau trên 120 test ngẫu nhiên mỗi bài (trừ những tầng được cố ý trình bày như một lời giải sai).

Nộp bài trên Luogu

- Đăng kí tài khoản tại luogu.com.cn (giao diện có thể chuyển sang tiếng Anh).
- Mở đề theo mã bài (ví dụ P3382 hoặc CF578C) bằng đường dẫn trong sách, chọn 提交答案 (nộp bài), chọn ngôn ngữ C++14 hoặc C++17 (bật O2 nếu muốn), dán code và nộp.
- Chương trình đọc từ bàn phím (stdin) và ghi ra màn hình (stdout).
- Với các bài mã CF, Luogu gửi bài nộp sang Codeforces chấm hộ (remote judge); cần liên kết tài khoản Codeforces theo hướng dẫn của Luogu.
- Kết quả: AC (đúng), WA (sai đáp án), TLE (quá thời gian), MLE (quá bộ nhớ), RE (lỗi khi chạy).

Bốn bước tư duy trước một bài tam phân



Bốn bước xây dựng lời giải một bài tam phân

- **Bước 1. Đặt hàm cần tối ưu.** Viết rõ f(x) và miền của x (đoạn số thực, tập số nguyên, số lượng k).
- **Bước 2. Chứng minh lồi hoặc một đỉnh.** Tổng và max của các hàm lồi là lồi; viết lại min(P, Q), |P − Q| để lộ tính lồi; khi không chứng minh được, ít nhất phải đối chiếu với vét cạn trên test nhỏ.
- **Bước 3. Chọn công cụ.** Tam phân; chặt nhị phân trên đạo hàm; chặt nhị phân trên giá trị (đối ngẫu); con trỏ khi điểm tối ưu đơn điệu; WQS khi lồi theo số lượng.
- **Bước 4. Kiểm soát sai số.** Lặp số lần cố định; cẩn thận với đoạn nằm ngang; in đúng đại lượng (giá trị hay vị trí).

Tình huống	Công cụ	Số lần tính hàm
f lồi trên đoạn số thực	tam phân	100 (khoảng còn $(2/3)^{100}$)
f trơn, biết đạo hàm	chặt nhị phân f'	60 – 100
f lồi trên số nguyên [1, V]	tam phân nguyên	$\approx 2 \cdot \log_{3/2} V$ (≈ 100 với $V = 10^9$)
f(x, y) lồi theo cặp	tam phân lồng nhau	100 × 100
g(k) lồi theo số lượng k	WQS	≈ log(miền λ) lần giải bài không ràng buộc

Bố cục sách

Chương	Chủ đề	Bài (mã Luogu)
1	Tam phân trên hàm số thực	P3382, P1883, CF578C
2	Tam phân trên số nguyên	CF439D, CF1355E, P3745
3	Tam phân lồng nhau, đối ngẫu và con trỏ	P2571, CF782B, CF939E
4	Tính lồi theo số lượng: chặt nhị phân hệ số phạt (WQS)	P2619, CF739E, P6246

Lê Thành Nam

Chương 1. Tam phân trên hàm số thực

Trọng tâm chương

Tam phân tìm đỉnh (hoặc đáy) của một hàm **một đỉnh** trên đoạn số thực. Chương xây dựng khuôn tam phân, cách nhận ra hàm lồi (max, tổng các hàm lồi), và mối liên hệ với chặt nhị phân trên đạo hàm.

Kiến thức cần nhớ

- Hàm một đỉnh: tăng rồi giảm (hoặc giảm rồi tăng). Hàm lồi là trường hợp quan trọng nhất.
- Cực tiểu: f(m1) > f(m2) ⇒ l = m1, ngược lại r = m2. Lặp cố định 100 lần.
- Tổng, max các hàm lồi là lồi; |hàm bậc nhất| là lồi.
- Hàm trơn một đỉnh: có thể chặt nhị phân trên dấu của đạo hàm.

Các bài trong chương

Bài	Tên bài	Mã Luogu	Nguồn	Số tầng
1	Tìm đỉnh của hàm một đỉnh (bài mẫu) 【模板】三分法	P3382	Bài mẫu (模板) của Luogu	3
2	Cực tiểu của hàm max (bài mẫu) 【模板】三分	函数	P1883	Bài mẫu (模板) của Luogu	2
3	Độ yếu và độ nghèo CF578C Weakness and Poorness	CF578C	Codeforces Round 320, bài C (hạng 1)	2

Lê Thành Nam

Bài 1. Tìm đỉnh của hàm một đỉnh (bài mẫu)

Luogu P3382 – 【模板】三分法

Nguồn: Bài mẫu (模板) của Luogu · Giới hạn thời gian sách dùng: 1,0 giây · Bộ nhớ: 128 MB
Đề gốc:https://www.luogu.com.cn/problem/P3382

Đề bài

Cho một đa thức bậc N và một đoạn [l, r]. Biết rằng trên [l, r], đa thức **tăng rồi giảm** (có đúng một đỉnh). Hãy tìm hoành độ x của đỉnh, chính xác tới 5 chữ số thập phân.

Dữ liệu vào

Dòng đầu chứa N, l, r. Dòng thứ hai chứa N + 1 hệ số theo thứ tự từ bậc cao xuống bậc thấp.

Kết quả

Hoành độ x của đỉnh, làm tròn 5 chữ số thập phân.

Ràng buộc

- 6 ≤ N ≤ 13 (ví dụ có N nhỏ hơn); các hệ số là số thực, |l|, |r| không lớn.

Ví dụ

Dữ liệu vào	Kết quả
3 -0.9981 0.5 1 -3 -3 1	-0.41421

Subtask

Các subtask dưới đây do sách bổ sung để luyện từng mức thuật toán; subtask cuối cùng là ràng buộc của đề gốc (hoặc mở rộng hơn đề gốc). Giới hạn thời gian 1,0 giây cho mọi subtask.

Subtask	Ràng buộc	Điểm
ST1	N ≤ 13 (gốc)	100%

Lê Thành Nam

Hướng dẫn giải

HIỂU ĐỀ Hiểu đúng đề bài

f(x) = $x^3 - 3x^2 - 3x + 1$ có đạo hàm $3x^2 - 6x - 3$, bằng 0 tại $x = 1 - \sqrt{2} \approx -0{,}41421$. Trên [−0,9981; 0,5], f tăng rồi giảm, đỉnh tại −0,41421.

Hàm "tăng rồi giảm" gọi là **hàm một đỉnh** (unimodal). Đây là điều kiện để dùng **tam phân** (三分法), giống như "đơn điệu" là điều kiện để chặt nhị phân.

NGÂY THƠ Ý tưởng ngây thơ: chia lưới

Tính f tại $2 \cdot 10^6$ điểm cách đều và lấy điểm lớn nhất. Độ chính xác bằng bước lưới $(r - l)/2 \cdot 10^6$.

Code. Tầng 1 – Chia lưới $2 \cdot 10^6$ điểm

```
#include <bits/stdc++.h>
using namespace std;
int N; double l, r;
vector<double> c;
double f(double x) { double s = 0; for (double v : c) s = s * x + v; return s; }   // Horner
int main() {
```

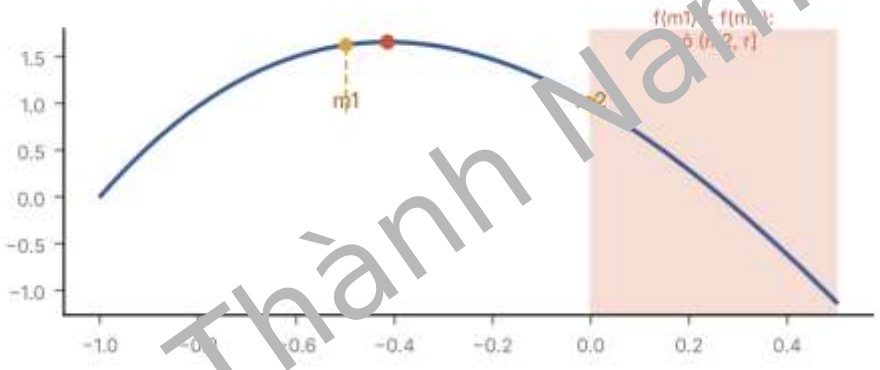
```
    scanf("%d %lf %lf", &N, &l, &r);
    c.resize(N + 1);
    for (auto &v : c) scanf("%lf", &v);
    const int K = 2000000;                    // chia [l, r] thành K đoạn, lấy điểm tốt nhất
    double best = l, bv = f(l);
    for (int i = 1; i <= K; i++) {
        double x = l + (r - l) * i / K, v = f(x);
        if (v > bv) { bv = v; best = x; }
    }
    printf("%.5f\n", best);
}
```

Với bài này cách làm vẫn đủ nhanh, nhưng độ chính xác tỉ lệ nghịch với số điểm: muốn chính xác thêm một chữ số phải tính gấp 10 lần.

> **Câu hỏi then chốt**
>
> Lấy hai điểm m1 < m2 trong [l, r]. Nếu f(m1) < f(m2), đỉnh có thể nằm trong [l, m1) không?

LÓE SÁNG Lóe sáng: so sánh hai điểm, bỏ một phần ba

Nếu f(m1) < f(m2) thì đỉnh **không thể** nằm bên trái m1: nếu đỉnh ở trái m1 thì f giảm trên [m1, m2], tức f(m1) ≥ f(m2), mâu thuẫn. Vậy bỏ [l, m1). Tương tự, nếu f(m1) ≥ f(m2) thì bỏ (m2, r].



Hình 1.1. *Chọn m1, m2 chia [l, r] thành ba phần; ở đây f(m1) ≥ f(m2) nên bỏ phần bên phải m2.*

Thiết kế lời giải – tam phân trên số thực	
Điều kiện	f một đỉnh (tăng rồi giảm) trên [l, r]
Lặp	m1 = l + (r − l)/3, m2 = r − (r − l)/3; f(m1) < f(m2) ⇒ l = m1; ngược lại r = m2
Số lần	cố định 100 (mỗi lần khoảng còn 2/3)

TỐI ƯU Lời giải tối ưu: tam phân

Code. Tầng 2 – Tam phân 100 lần

```
#include <bits/stdc++.h>
using namespace std;
int N; double l, r;
vector<double> c;
double f(double x) { double s = 0; for (double v : c) s = s * x + v; return s; }
int main() {
    scanf("%d %lf %lf", &N, &l, &r);
    c.resize(N + 1);
    for (auto &v : c) scanf("%lf", &v);
    for (int it = 0; it < 100; it++) {        // tam phân: mỗi lần bỏ 1/3 khoảng
        double m1 = l + (r - l) / 3, m2 = r - (r - l) / 3;
        if (f(m1) < f(m2)) l = m1;            // đỉnh không nằm trong [l, m1)
        else r = m2;                          // đỉnh không nằm trong (m2, r]
    }
    printf("%.5f\n", (l + r) / 2);
}
```

Lê Thành Nam